

NOM	
Prénom	
Date de naissance	

Documents à compléter et à rendre

Description	Valeur
Lien git front end	https://github.com/achassefiere/projet_final_studi
Lien git back end	https://github.com/achassefiere/projet_final_studi
Lien application	https://antoinec.pythonanywhere.com/
Login admin	antoinec
mot de passe admin	123+Aze!

- 1. Concevez une documentation technique, mettant en avant :**
 - a. La préparation de votre git ainsi que la gestion de vos branches
 - b. Sous forme de “bullet points”, mettez en évidence votre démarche pour développer une user Story
 - c. Mentionnez les outils et technologies qui ont été utilisés
 - d. Développez toutes les user story avec les bonnes pratiques. Il est important de noter que tous les user stories déployés seront en statut “done”
 - e. Produisez des tests unitaires (80% de votre application doit être testé)

TOUT EST DISPONIBLE SUR LE GIT (DOCUMENTATION TECHNIQUE, README, PROJET)

1. Déployez toutes vos user stories, puis, montrez et justifiez les mesures de sécurité qui sont mises en place.

Toutes les User Stories ont été mises en place sur le site dont le lien est en en-tête du projet.

Au niveau des mesures de sécurité, lorsqu'un client s'inscrit, il doit rentrer son mot de passe deux fois dans le formulaire, de plus, ce mot de passe est hashé dans la base de données. Il ne doit pas y apparaître en clair.

Pour la majorité des view nécessaires, correspondant a des fonctions de l'application, @login_required et @user_pass_test(is_admin) ont été codées en entête ainsi que des balises `{% if user.is_authenticated %} {% if user.is_staff %} {% if user.is_staff %}` ont été codées afin qu'un utilisateur non inscrit, ou n'étant pas administrateur ne puisse pas avoir accès aux ajout, modifications et suppressions de véhicules, valider et rejeter dossier (admin) ou créer et supprimer dossier (client).

2. Comment surveillez-vous votre application (logs, gestion d'erreur). Implémentez une solution d'alerting

Django est configuré avec un système de logs dans le fichier settings.py.

Les événements importants (erreurs, avertissements, informations) sont enregistrés dans des fichiers de logs et/ou affichés dans la console selon l'environnement d'exécution.

L'application utilise également l'outil Sentry pour surveiller les erreurs en temps réel.

Sentry est connecté à Django et collecte automatiquement :

- les exceptions non gérées ;

- les erreurs serveur (HTTP 500) ;

- les traces complètes d'exécution (stack traces) ;

- le contexte de la requête concernée.

Cela permet d'identifier rapidement l'origine d'un problème sans devoir analyser manuellement les fichiers de logs.

L'alerting est également géré par Sentry, lorsqu'une erreur critique survient, un mail est alors envoyé au développeur et les informations techniques nécessaires au diagnostic de l'erreur sont disponibles immédiatement.

Cela nous permet de garder constamment un œil sur l'application.